djkurbaga / BIL481-2026-Project

Code Pull requests Agents Actions Projects Wiki Security and quality Insights

Settings

main BIL481-2026-Project / docs / project-definition.md

BulentCakirr Define project success criteria for benchmarks 7b66f1b · 24 minutes ago

163 lines (117 loc) · 12.5 KB

Project Definition Document

Project: Green Software Metrics Framework (GSMF) **Course:** BIL 481 Software Engineering, Fall 2026 **Group:** Intellectual Property **Document date:** 01-11-2026

Contributors to this document: Reşat Efe Koca (231301010), Bülent Çakır (231301064), Murat Yılmaz (241101063), Furkan Yaman (221301015)

Table of Contents

1. Project Name
2. Project Summary
3. Objectives
4. Scope
5. Target Audience
6. Key Features
7. Deliverables
8. Budget and Resources
9. Risks and Mitigation Strategies
10. Project Success Criteria
11. Document-Specific Task Matrix

---

1. Project Name

Green Software Metrics Framework (GSMF) — an energy and efficiency benchmarking harness for low-code platforms.

2. Project Summary

Low-code platforms let teams build applications quickly, but the runtime energy cost of their abstraction layers is rarely measured. Developers and organizations therefore have no reliable way to tell how energy-efficient a low-code application is compared to a hand-coded equivalent.

GSMF is a command-line benchmarking framework that measures and compares the energy consumption and efficiency of the same reference application built on self-hosted low-code platforms (Budibase and Appsmith, running in Docker) and on a hand-coded FastAPI baseline. It reads CPU energy directly from Linux RAPL counters, drives a standardized workload from a separate load-generator machine, applies controlled experimental conditions, and reports energy per user task, Energy-Delay Product (EDP) and the operational Software Carbon Intensity (SCI) score defined by the Green Software Foundation.

3. Objectives

By the end of the project we aim to:

1. Implement the same reference CRUD application (create item, paginated list, batch update) on Budibase, Appsmith and FastAPI.
2. Build a Python CLI that runs one platform stack at a time, measures idle and workload energy via RAPL, and computes net energy.
3. Run at least 10 repetitions per platform under controlled conditions (fixed CPU governor, Turbo Boost off, cooldown between runs).
4. Report mean energy, execution time, EDP and operational SCI per completed user task, with 95% confidence intervals.
5. Provide a simple results dashboard and CSV/report export so results can be compared and reproduced.

4. Scope

Included

- Self-hosted, Docker-based deployments of Budibase and Appsmith, plus a FastAPI + SQLite baseline.

- One standardized reference application and workload, driven by k6 from a second machine.
- CPU package (and DRAM where available) energy measurement through Linux RAPL sysfs.
- Idle baseline subtraction, repeated trials, statistical aggregation and operational SCI calculation.
- CLI tool, CSV/Markdown reports and a lightweight web dashboard.
- Optional per-container breakdown with Scaphandre as a secondary diagnostic.

Not included

- Vendor-hosted SaaS/cloud low-code platforms (e.g., Bubble, OutSystems Cloud), because their servers cannot be instrumented.
- Browser/frontend rendering energy measurement.
- Embodied carbon (the M term of SCI); documented as future work.
- Converting network traffic into energy; network data is recorded only as secondary telemetry.
- Kubernetes deployments, external hardware power meters, and non-x86 hardware.

In scope:

- Functionally equivalent reference applications on Budibase, Appsmith and FastAPI, implementing item creation, paginated listing and batch updates with comparable datasets and workload semantics.
- A Python CLI with YAML-based configuration, isolated Docker Compose stack management and remote k6 workload execution from a separate load-generator machine.
- Linux RAPL measurements on the testbed, including idle baseline correction and repeated trials under controlled CPU settings with cooldown intervals.
- Statistical analysis and calculation of energy per completed user task, EDP and operational SCI within the declared RAPL measurement boundary.
- SQLite result storage, CSV and Markdown exports, a simple comparison dashboard, and documentation of the environment and procedure needed to reproduce the experiments.

Out of scope:

- Vendor-hosted SaaS platforms, additional low-code platforms and Kubernetes deployments.
- Browser/frontend and load-generator energy, network-to-energy conversion, embodied carbon and external hardware power-meter measurements.
- Production deployment, production-scale capacity planning and general claims about all applications built on the evaluated platforms.

The system boundary includes the benchmark controller, platform stacks, measurement and analysis modules, and reporting components. The remote load generator supplies the workload; its energy consumption is outside the reported measurement boundary. Results describe the selected reference application and test environment, rather than whole-system electricity consumption.

5. Target Audience

- Software developers and citizen developers choosing between low-code platforms and hand-coding.
- IT and sustainability managers who need evidence about the energy cost of software choices.
- Students and researchers working on green software and software measurement.

Audience	Intended Use
Software developers and architects	Compare the measured runtime energy and execution-time tradeoffs of the reference application across low-code platforms and a hand-coded baseline.
Software engineering researchers and students	Reproduce controlled experiments, inspect raw measurements and study variability and limitations in software energy benchmarking.
Test and performance engineers	Configure workloads, execute repeatable benchmark runs and investigate differences under consistent test conditions.
Engineering teams evaluating low-code adoption	Use the comparative report as supporting evidence when assessing platform choices for similar workloads, alongside functional and organizational requirements.

The CLI is intended for technical users familiar with Linux, Docker and basic benchmark configuration. The dashboard and exported reports provide a more accessible view for stakeholders reviewing the findings.

6. Key Features

1. **Benchmark Configuration Management:** Define the target platforms, workload size, concurrency level, number of repetitions and cooldown duration in a YAML configuration file. Validate configuration parameters before execution to support consistent and reproducible experiments.
2. **Isolated Benchmark Environment Management:** Automate the Docker stack lifecycle, including startup, health checks and shutdown. Run one platform stack at a time to minimize interference between benchmark executions.

3. **Energy Measurement and Baseline Correction:** Collect Linux RAPL energy-counter readings during idle and workload phases. Calculate net workload energy by subtracting the idle baseline contribution for the corresponding workload duration.
4. **Remote Workload Orchestration:** Trigger the standardized k6 workload on a separate load-generator machine. Record workload completion, execution time and completed user-task counts to support comparable measurements across platforms.
5. **Statistical Results Analysis:** Aggregate repeated benchmark measurements and calculate the mean, median, standard deviation and 95% confidence intervals. Use these statistics to describe measurement variability and support platform comparisons.

7. **Results Reporting and Visualization:** Present benchmark summaries through CLI tables, export results in CSV and Markdown formats, and provide a simple dashboard for comparing platforms and their measured performance and energy efficiency.

7. Deliverables

Deliverable	Contents and Purpose
Software Engineering Documents (Assignment 1)	Project Definition, Software Requirements and Project Plan documents describing the project scope, expected system behavior, responsibilities and planned work.
Reference Application Implementations	Functionally equivalent reference CRUD applications on Budibase, Appsmith and FastAPI, covering item creation, paginated listing and batch updates. Include Docker Compose files for deployment and k6 scripts for the standardized workload.
GSMF CLI Source Code	The Python benchmarking framework maintained in the GitHub repository, including configuration management, stack lifecycle management, energy measurement, workload orchestration, statistical analysis and metric calculation. Include setup and execution instructions and an example YAML configuration.
Benchmark Dataset	Raw benchmark results in CSV format, accompanied by the configurations and experimental conditions needed to interpret and reproduce the measurements.
Comparative Evaluation Report	A comparison of Budibase, Appsmith and FastAPI based on repeated benchmark runs, including descriptive statistics, 95%

Deliverable	Contents and Purpose
	confidence intervals, green software metrics, calculation assumptions and measurement limitations.
Results Dashboard	A simple interface for reviewing benchmark summaries and comparing energy consumption, execution time and derived metrics across the evaluated platforms.
Final Documentation and Presentation	Documentation of the implemented system, its setup and usage, the experimental method and the findings, together with the final project presentation.

8. Budget and Resources

- **Team:** Four members (roles listed in the Project Plan).
- **Software (free/open source):** Python 3.11, FastAPI, SQLite, Docker and Docker Compose, Budibase and Appsmith community editions, k6, Scaphandre, Git/GitHub.
- **Hardware:** One Intel-based x86 machine running Ubuntu as the testbed (a desktop or mini PC preferred; a laptop on AC power is the fallback), one team laptop as the load generator, and a local network switch or router.
- **Learning resources:** Green Software Foundation SCI specification, Pereira et al. (2017), platform and tool documentation.
- **Time:** One semester (October 2026 – January 2027).
- **Budget:** No paid services are required; all tools are free or open source and run on team-owned hardware. Expected cost is 0 TL.

9. Risks and Mitigation Strategies

Risk	Impact	Mitigation
RAPL not available or blocked on the testbed hardware	Measurements impossible	Verify RAPL access in week 1; keep a second team machine as backup testbed
Measurement noise from thermal throttling or background processes	Unreliable results	Fix CPU governor, disable Turbo Boost, cooldown between runs, headless OS, 10+ repetitions
Building the same app on three platforms takes longer than planned	Schedule delay	Keep the reference app to three operations; build FastAPI baseline first

Risk	Impact	Mitigation
Low-code platform setup or version issues in Docker	Blocked experiments	Pin platform versions in Compose files; document setup steps
Team member unavailability	Lost capacity	Shared task matrix, pair ownership of critical modules, weekly status check
Scope creep in the dashboard	Less time for measurement	CLI first; dashboard only after measurement pipeline is stable

10. Project Success Criteria

1. The reference application runs successfully on all three targets and completes 100% of workload tasks in each benchmark run.
2. At least 10 repetitions per platform are collected, with a 95% confidence interval half-width below 10% of the mean net energy.
3. All seven key features are implemented and demonstrated end to end with a single CLI command.
4. Another team member can reproduce a full benchmark from the repository instructions on the testbed without code changes.
5. The final report presents energy per task, EDP and operational SCI for all three platforms.

11. Document-Specific Task Matrix

Task	Responsible
Project name, summary and objectives	Reşat Efe Koca
Scope and target audience	Murat Yılmaz
Key features and deliverables	Bülent Çakır
Budget, resources, risks and mitigation	Furkan Yaman
Success criteria	Bülent Çakır
Review, formatting and final PDF	Reşat Efe Koca